

UNIVERSIDAD AUTÓNOMA
METROPOLITANA

Unidad Cuajimalpa

PROYECTO FINAL

Sistemas Distribuidos

Mini Plataforma de Video Streaming P2P

Presenta:

Lizeth Sofía Vázquez Rivera

Michelle Morán Cruz

Enrique García Ortega

Profesor:

Dr. Guillermo Monroy Rodríguez

Fecha de entrega:

13 de Julio 2026

Índice

1. Introducción	2
2. Creación de carpetas	2
3. División del video	4
4. Exposición de una API REST	5
5. Configuración del FragmentController en el Nodo A.	6
6. Configuración del FragmentController en el Nodo B.	7
7. Configuración del FragmentController en el Nodo C.	8
8. Comunicación entre Nodo A y Nodo B.	8
9. Descarga de fragmentos.	9
10.Comunicación entre Nodo C y Nodo B.	10
11.Prueba de los Endpoints REST.	10
12.Configuración de RabbitMQ.	12
13.Uso de Docker	13
14.Comunicación Publish/Subscribe.	13
15.Pruebas Finales	14
16.Conclusión	16

1. Introducción

En la actualidad, los sistemas distribuidos se han convertido en una de las principales soluciones para desarrollar aplicaciones escalables, flexibles y capaces de trabajar de manera colaborativa a través de múltiples equipos o servicios. A diferencia de los sistemas tradicionales, un sistema distribuido divide las tareas entre diferentes nodos que se comunican entre sí mediante redes y protocolos de comunicación, permitiendo compartir recursos y mejorar el rendimiento del sistema.

El presente proyecto tiene como propósito desarrollar un sistema distribuido basado en una arquitectura de microservicios utilizando Spring Boot, Docker, RabbitMQ y APIs REST. El sistema implementa tres nodos independientes (Nodo A, Nodo B y Nodo C), los cuales colaboran para almacenar y compartir fragmentos de un video. Para ello, el archivo original se divide en varias partes, permitiendo que cada nodo conserve únicamente algunos fragmentos y pueda intercambiarlos con los demás cuando sea necesario.

La comunicación entre los nodos se realiza mediante servicios REST, mientras que RabbitMQ se utiliza como un sistema de mensajería bajo el modelo Publish/Subscribe para enviar notificaciones entre los microservicios. Asimismo, Docker facilita la ejecución y administración de los diferentes componentes del sistema dentro de contenedores, garantizando un entorno uniforme para su funcionamiento.

Durante el desarrollo del proyecto se llevaron a cabo diversas pruebas para verificar la correcta comunicación entre los nodos, el funcionamiento de las APIs REST, la configuración de RabbitMQ, el intercambio de fragmentos y la descarga del video solicitado. Los resultados obtenidos demuestran que la arquitectura implementada permite la colaboración entre los diferentes servicios y cumple con los principios fundamentales de los sistemas distribuidos.

2. Creación de carpetas

En esta etapa se organizó la estructura del proyecto para implementar una arquitectura basada en microservicios. Se creó un nodo independiente para cada servicio (Nodo A, Nodo B y Nodo C), permitiendo que cada uno funcionara de manera autónoma y pudiera almacenar una parte del video.

Cada nodo cuenta con su propia estructura de proyecto Spring Boot, incluyendo sus controladores, servicios, recursos y configuraciones necesarias

para su ejecución.

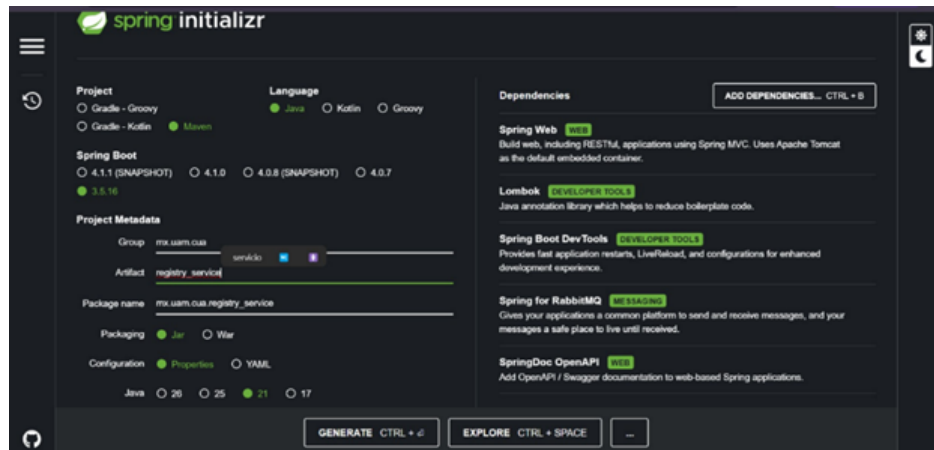


Figura 1: Imagen: 1

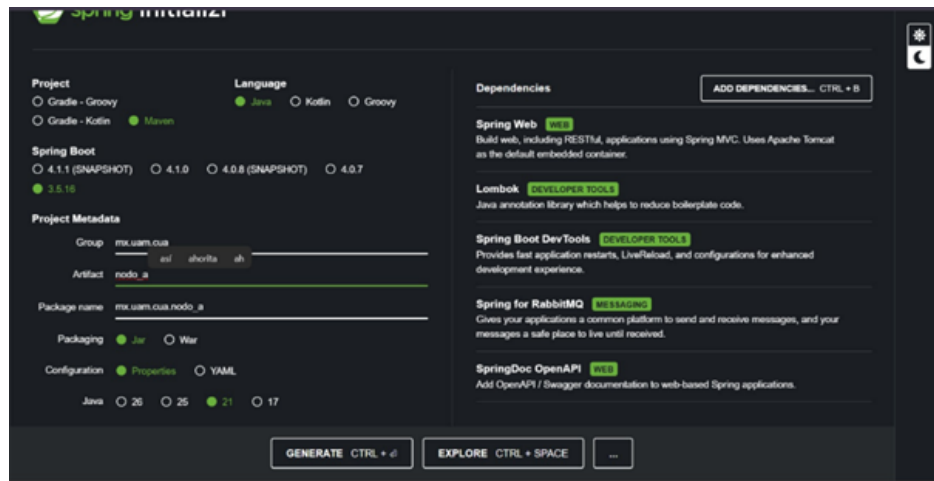


Figura 2: Imgen: 2

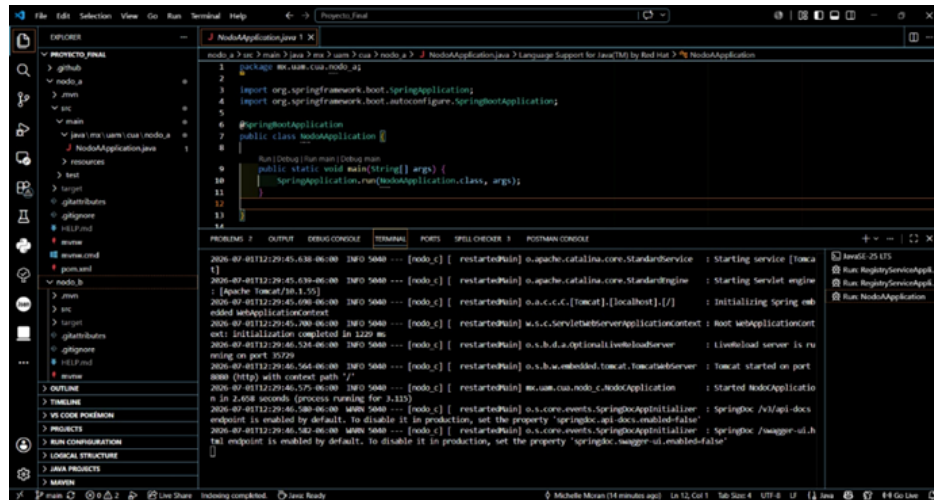


Figura 3: Imgen: 3

3. División del video

Se tomó un video de prueba y se dividió en 10 fragmentos pequeños.

El objetivo de esta división es que ningún nodo tenga que almacenar el archivo completo, sino únicamente una parte de él. Esto reduce la carga de almacenamiento y facilita el intercambio de información entre los nodos.

Cada fragmento puede ser solicitado posteriormente por cualquier nodo que lo requiera.

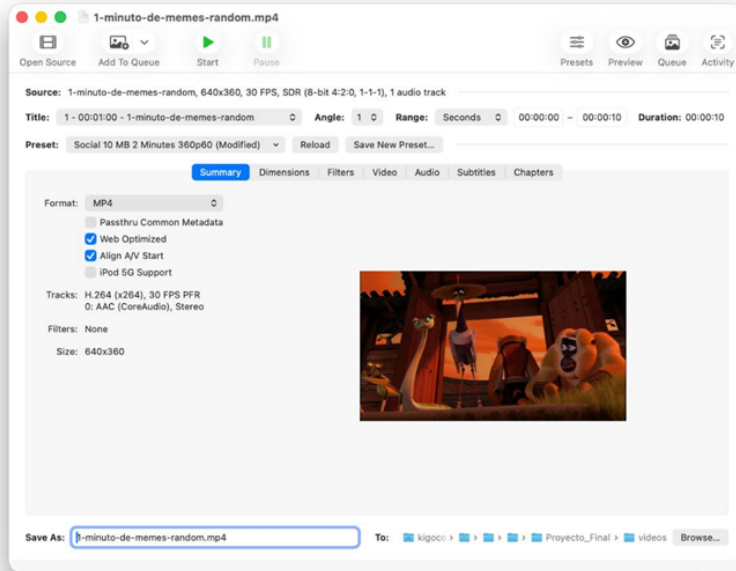


Figura 4: Imagen: 4

4. Exposición de una API REST

Cada nodo implementa una API REST desarrollada con Spring Boot. Estas APIs permiten realizar operaciones como:

- Consultar los videos disponibles.
- Descargar un fragmento específico.
- Compartir información con otros nodos.
- Solicitar recursos almacenados en otro nodo.

La comunicación se realiza mediante el protocolo HTTP utilizando solicitudes GET y POST.

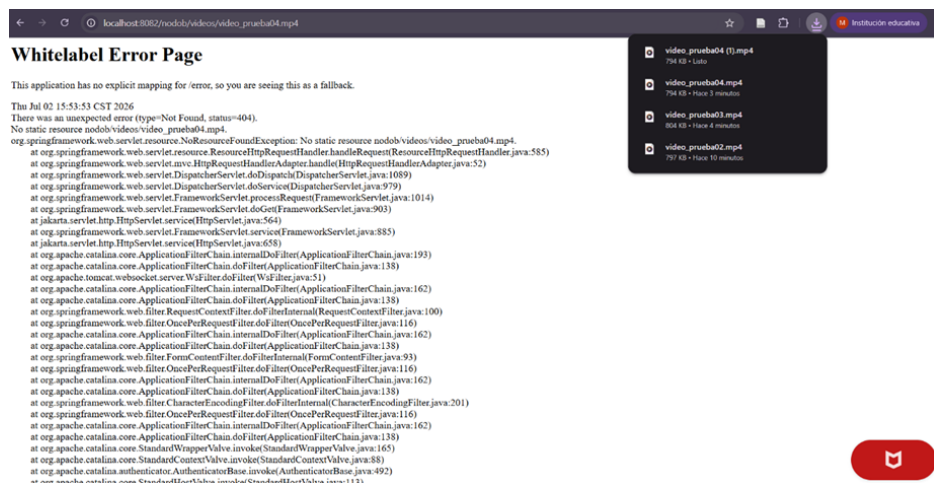


Figura 5: Imagen: 5

5. Configuración del FragmentController en el Nodo A.

Después de modificar el controlador de fragmentos del Nodo A, se realizaron pruebas para verificar que el servicio respondiera correctamente a las solicitudes realizadas desde otros nodos. Los resultados obtenidos demostraron que el controlador podía listar correctamente los videos disponibles y responder a las peticiones REST sin errores.

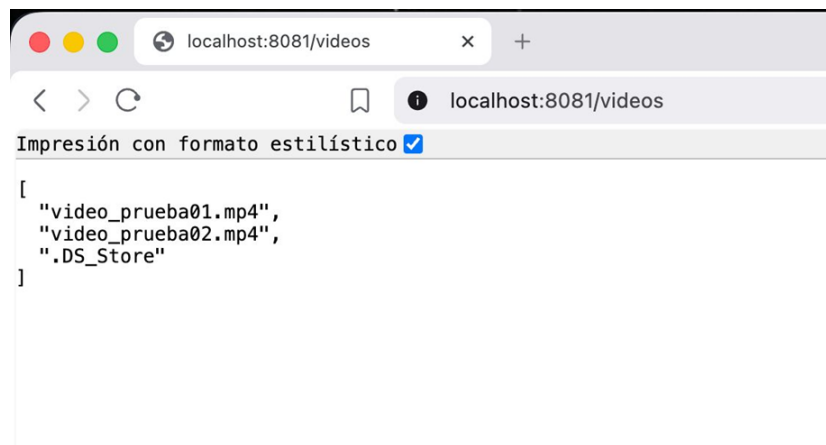


Figura 6: Imagen: 6

6. Configuración del FragmentController en el Nodo B.

Se realizaron modificaciones similares en el Nodo B. Posteriormente se comprobó que este nodo pudiera atender solicitudes provenientes del Nodo A y compartir correctamente los fragmentos almacenados localmente.

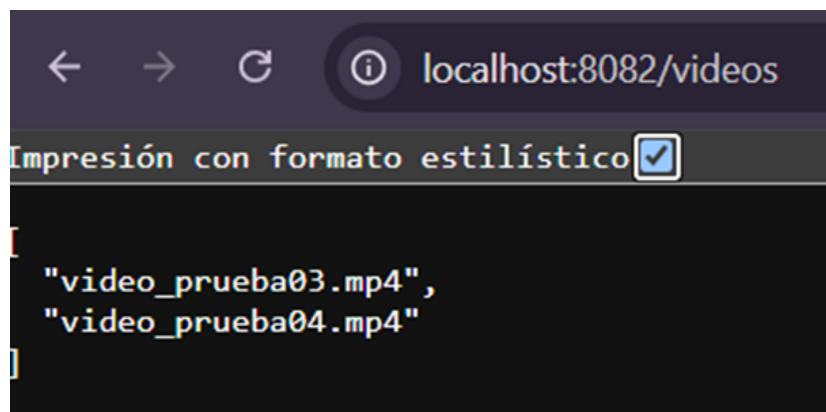


Figura 7: Imagen: 7

7. Configuración del FragmentController en el Nodo C.

Finalmente se implementó la misma funcionalidad en el Nodo C. Con esto los tres nodos quedaron preparados para responder solicitudes REST y participar en el intercambio de fragmentos del video.

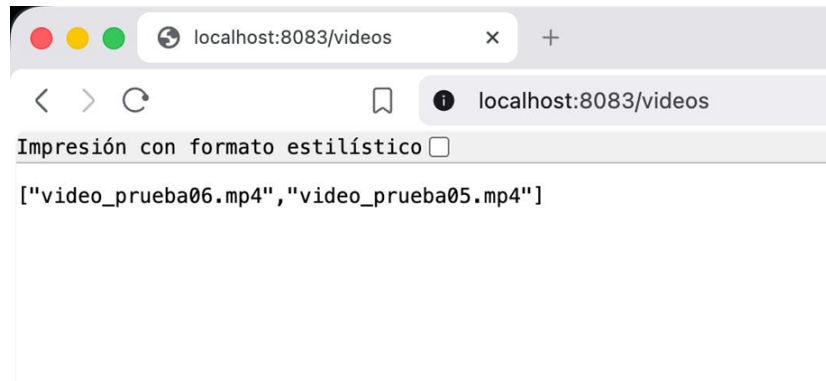


Figura 8: Imagen: 8

8. Comunicación entre Nodo A y Nodo B.

Se realizaron pruebas enviando solicitudes desde el puerto 8081 hacia el puerto 8082 y viceversa. Estas pruebas permitieron verificar que ambos nodos podían consultar la lista de videos disponibles y establecer comunicación mediante las APIs REST implementadas. Los resultados confirmaron una comunicación correcta entre ambos microservicios.

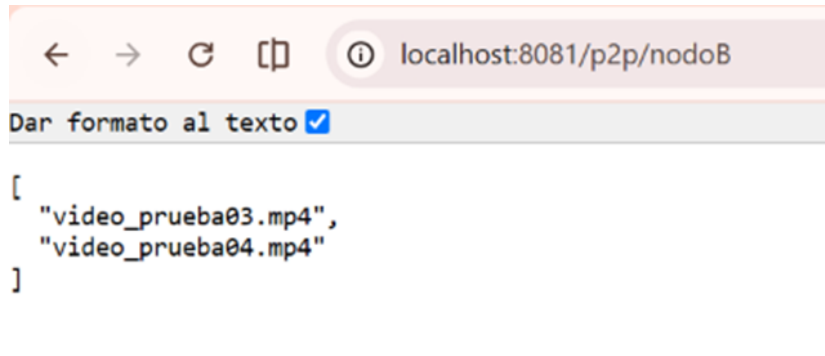


Figura 9: Imagen: 9

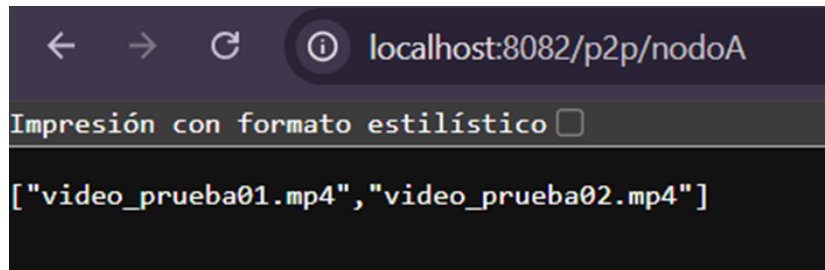


Figura 10: Imagen: 10

9. Descarga de fragmentos.

Una vez establecida la comunicación entre los nodos, se realizaron pruebas de descarga. El Nodo B solicitó un fragmento disponible en otro nodo utilizando la API REST correspondiente. La descarga se realizó correctamente, demostrando el funcionamiento del intercambio de archivos entre microservicios.



Figura 11: Imagen: 11

10. Comunicación entre Nodo C y Nodo B.

También se realizaron pruebas entre el Nodo C y el Nodo B para validar que cualquier nodo pudiera comunicarse con otro independientemente de su ubicación. Durante esta prueba se consultaron los videos disponibles y se verificó la correcta respuesta del servidor remoto.

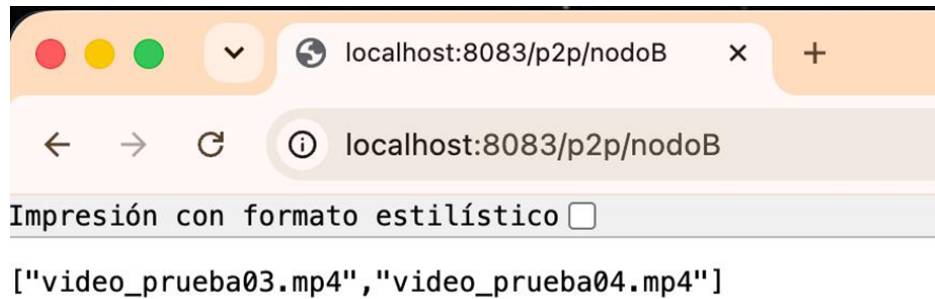


Figura 12: Imagen: 12

11. Prueba de los Endpoints REST.

Se probaron diferentes endpoints del sistema, entre ellos:

- /videos
- /videos/video_prueba01.mp4
- /p2p/nodoB

Estas pruebas permitieron comprobar que los controladores REST respondían correctamente a las solicitudes HTTP realizadas por los demás nodos.

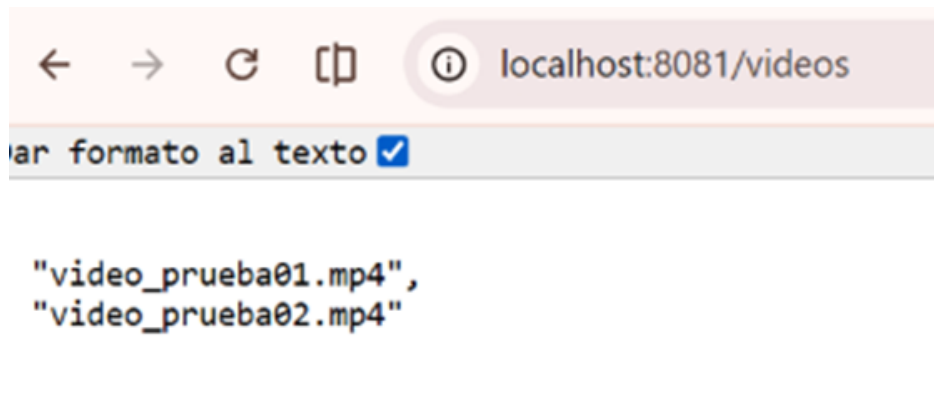


Figura 13: Imagen: 13

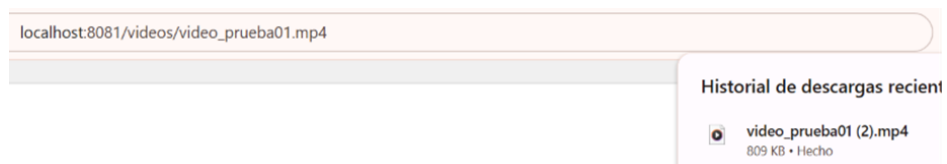


Figura 14: Imagen: 14

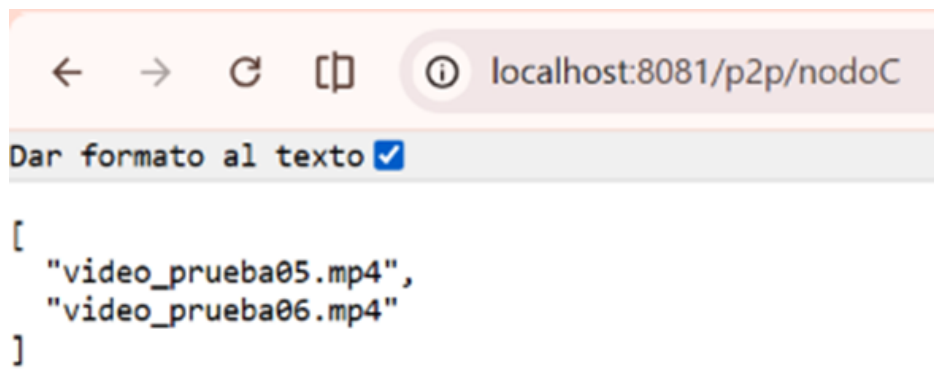


Figura 15: Imagen: 15

12. Configuración de RabbitMQ.

Se implementó RabbitMQ como sistema de mensajería entre los nodos. RabbitMQ permite enviar mensajes de tipo Publish/Subscribe (Pub/Sub), facilitando la comunicación asíncrona entre los microservicios sin necesidad de mantener una conexión directa permanente. Para ello se configuraron:

- Usuario
- Contraseña
- Host
- Puerto de comunicación

Además, se verificó el correcto inicio del servidor RabbitMQ utilizando Docker.

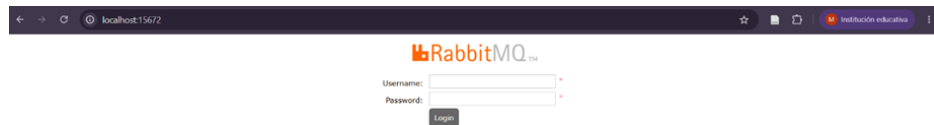


Figura 16: Imagen: 16

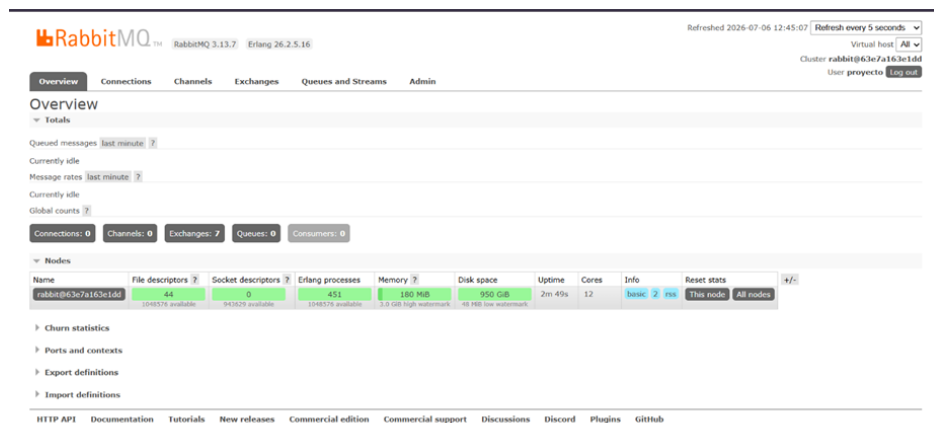


Figura 17: Imagen: 17

13. Uso de Docker

Todos los servicios fueron ejecutados mediante Docker. Con ayuda del archivo docker-compose.yml fue posible iniciar RabbitMQ y los demás componentes del sistema de manera sencilla. También se utilizó el comando docker ps para comprobar que todos los contenedores estuvieran activos y funcionando correctamente.

```
4 files changed, 4 insertions(+), 4 deletions(-)
C:\Users\miche\OneDrive\Escritorio\MICH UAM\Sistemas Distribuidos\Proyecto_Final>docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
63e7a163e1dd   rabbitmq:3-management               "docker-entrypoint.s..." 47 hours ago   Up 21 seconds  0.0.0.0:5672->5672/tcp, [::]:5672->5672/tcp
, 0.0.0.0:15672->15672/tcp, [::]:15672->15672/tcp   rabbitmq
C:\Users\miche\OneDrive\Escritorio\MICH UAM\Sistemas Distribuidos\Proyecto_Final>
```

Figura 18: Imagen: 18

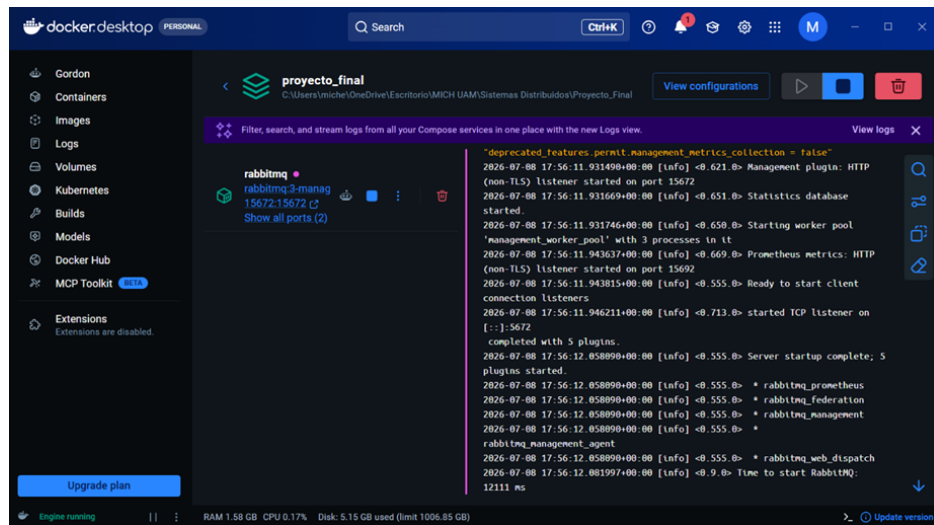


Figura 19: Imagen: 19

14. Comunicación Publish/Subscribe.

Se realizaron pruebas utilizando RabbitMQ para comprobar la comunicación mediante el modelo Publish/Subscribe. Cuando un nodo publica un mensaje indicando la disponibilidad de un fragmento, los demás nodos reci-

ben la notificación automáticamente. Esto demuestra el funcionamiento de la comunicación asíncrona entre microservicios.

```
Guardando en: C:\Users\miche\OneDrive\Escritorio\MICH UAM\Sistemas Distribuidos\Proyecto_Final\nodo_a\videos\curiosidades04.mp4
Archivo descargado: curiosidades04.mp4
Mensaje enviado:
Nodo: nodo_a
Video: curiosidades04.mp4
Fragmento: 1
=====
Mensaje recibido
Nodo: nodo_a
Video: curiosidades04.mp4
Fragmento: 1
=====
Guardando en: C:\Users\miche\OneDrive\Escritorio\MICH UAM\Sistemas Distribuidos\Proyecto_Final\nodo_a\videos\curiosidades04.mp4
Archivo descargado: curiosidades04.mp4
Mensaje enviado:
Nodo: nodo_a
Video: curiosidades04.mp4
Fragmento: 1
=====
Mensaje recibido
Nodo: nodo_a
Video: curiosidades04.mp4
Fragmento: 1
=====
C:\Users\miche\OneDrive\Escritorio\MICH UAM\Sistemas Distribuidos\Proyecto_Final>
```

Figura 20: Imagen: 20

15. Pruebas Finales

Finalmente se realizaron pruebas integrales del sistema. Durante estas pruebas se verificó:

- La comunicación entre los tres nodos.
- El intercambio de fragmentos.
- La respuesta correcta de las APIs REST.
- El envío de mensajes mediante RabbitMQ.
- La descarga correcta de los archivos de video.

Los resultados obtenidos demostraron que la arquitectura distribuida implementada cumple con los objetivos establecidos para el proyecto.

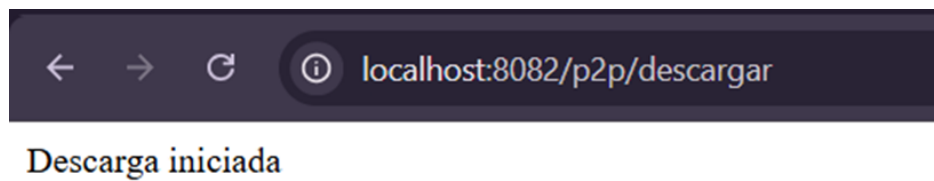


Figura 21: Imagen: 21

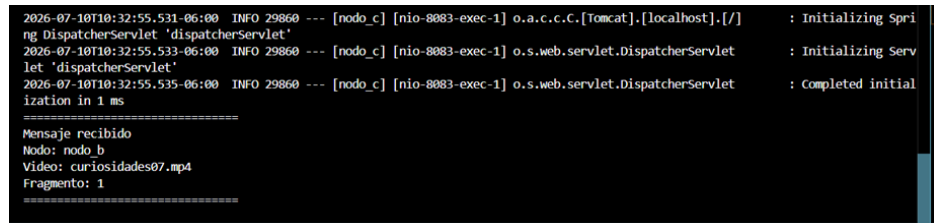


Figura 22: Imagen: 22

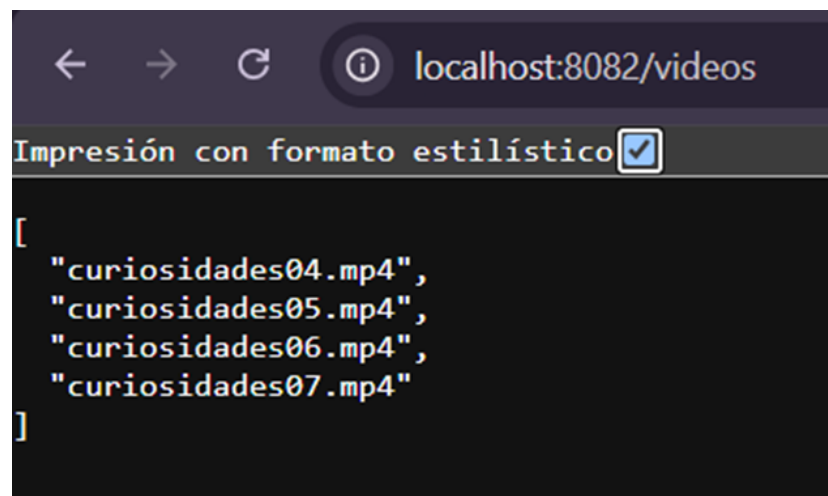


Figura 23: Imagen: 23

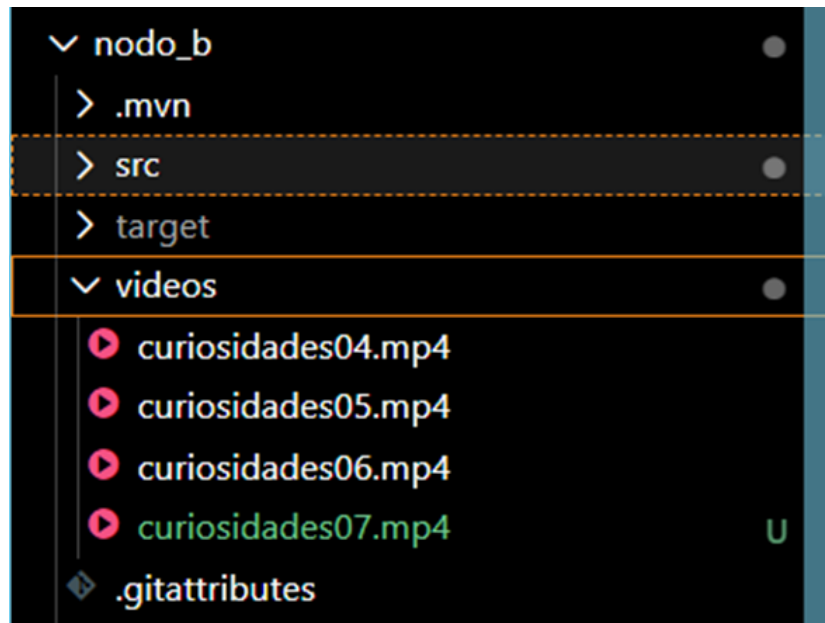


Figura 24: Imagen: 24

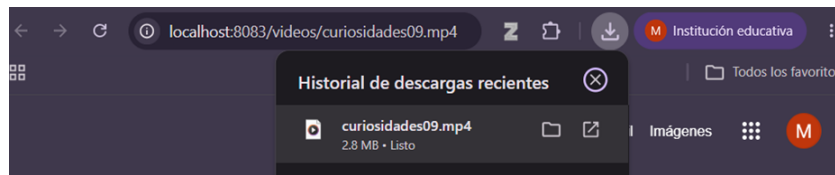


Figura 25: Imagen: 25

16. Conclusión

El proyecto implementa una arquitectura de sistemas distribuidos basada en microservicios desarrollados con Spring Boot. Cada nodo almacena únicamente una parte del video y puede compartirla con los demás mediante APIs REST. La comunicación asíncrona se realiza con RabbitMQ utilizando el modelo Publish/Subscribe, mientras que Docker facilita el despliegue de todos los servicios. Las pruebas realizadas demostraron que los nodos pueden intercambiar fragmentos, comunicarse correctamente y reconstruir el contenido solicitado, cumpliendo así con los objetivos planteados para el proyecto.